

Week3

JWT-handson

| src

JwtHandsonApplication.java

```
package com.cognizant;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JwtHandsonApplication {

    public static void main(String[] args) {
        SpringApplication.run(JwtHandsonApplication.class, args);
    }

}
```

SecurityConfig.java

```
package com.cognizant.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder; import
    org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {
```

```

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("pwd"))
            .roles("ADMIN")
            .build();

        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder().encode("pwd"))
            .roles("USER")
            .build();

        return new InMemoryUserDetailsManager(admin, user);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http

            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/authenticate")
                .hasAnyRole("USER", "ADMIN")
                .anyRequest()
                .authenticated())
            .httpBasic(Customizer.withDefaults());

        return http.build();
    }
}

```

AuthenticationController.java

```
package com.cognizant.controller;
```

```

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@RestController
public class AuthenticationController {

    private static final String SECRET =
        "secretkeysecretkeysecretkey12345";

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {

        Map<String, String> map = new HashMap<>();

        String user = getUser(authHeader);

        String token = generateJwt(user);

        map.put("token", token);

        return map;
    }

    private String getUser(String authHeader) {

        String encodedCredentials =
            authHeader.substring("Basic ".length());

```

```

byte[] decodedBytes =
Base64.getDecoder().decode(encodedCredentials);

String credentials = new String(decodedBytes);

return credentials.substring(0, credentials.indexOf(":"));
    }

private String generateJwt(String user) {

    SecretKey key = Keys.hmacShaKeyFor(
        SECRET.getBytes(StandardCharsets.UTF_8));

    return Jwts.builder()
        .subject(user)
        .issuedAt(new Date())
        .expiration(new Date(System.currentTimeMillis() + 1200000))
        .signWith(key)
        .compact();
    }
}

```

application.properties

```

spring.application.name=JWT-handson

spring.security.user.name=user
    spring.security.user.password=password

```

JwtHandsonApplicationTests.java

```

package com.cognizant;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class JwtHandsonApplicationTests {

    @Test
    void contextLoads() {
    }
}

```

```
)
```

| target

application.properties

```
spring.application.name=JWT-handson

spring.security.user.name=user
    spring.security.user.password=password
```

JwtHandsonApplication.class

```
[Binary file - content not shown]
```

SecurityConfig.class

```
[Binary file - content not shown]
```

AuthenticationController.class

```
[Binary file - content not shown]
```

JwtHandsonApplicationTests.class

```
[Binary file - content not shown]
```

Spring Rest using Spring Boot

| spring-rest-handson

mvnw

```
#!/bin/sh

# -----

# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements. See the NOTICE
# file distributed with this work for additional information
# regarding copyright ownership. The ASF licenses this
# file to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License. You may obtain a copy of the License at
#
#
# http://www.apache.org/licenses/LICENSE-2.0 #
# Unless required by applicable law or agreed to in writing,
```

```

# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.
# -----

# -----

# Apache Maven Wrapper startup batch script, version
3.3.4 #
# Optional ENV vars
# -----
#   JAVA_HOME - location of a JDK home dir, required when download maven via java
#               source
#   MVNW_REPOURL - repo url base for downloading maven distribution
#   MVNW_USERNAME/MVNW_PASSWORD - user and password for downloading maven
#   MVNW_VERBOSE - true: enable verbose log; debug: trace the mvnw script;
#                   others: silence the output
# -----

set -euf
[ "${MVNW_VERBOSE-}" != debug ] || set -x

# OS specific support.
native_path() { printf %s\\n "$1"; }
case "$(uname)" in
  CYGWIN* | MINGW*)
    [ -z "${JAVA_HOME-}" ] || JAVA_HOME="$(cygpath --unix "$JAVA_HOME")"
    native_path() { cygpath --path --windows "$1"; }
    ;;
esac

# set JAVACMD and JAVACCMD
set_java_home() {
  # For Cygwin and MinGW, ensure paths are in Unix format before anything is
  # touched
  if [ -n "${JAVA_HOME-}" ]; then
    if [ -x "$JAVA_HOME/jre/sh/java" ]; then
      # IBM's JDK on AIX uses strange locations for the executables
      JAVACMD="$JAVA_HOME/jre/sh/java"
    fi
  fi
}

```

```

JAVACCMD="$JAVA_HOME/jre/sh/javac"
else
JAVACMD="$JAVA_HOME/bin/java"
JAVACCMD="$JAVA_HOME/bin/javac"

if [ ! -x "$JAVACMD" ] || [ ! -x "$JAVACMD" ]; then
echo "The JAVA_HOME environment variable is not defined correctly, so
mvnw cannot run." >&2
echo "JAVA_HOME is set to \"$JAVA_HOME\", but \"$JAVA_HOME/bin/java\" or
\"$JAVA_HOME/bin/javac\" does not exist." >&2
return 1
fi
fi
else
JAVACMD="$(
'set' +e
'unset' -f command 2>/dev/null
'command' -v java
)" || :
JAVACMD="$(
'set' +e
'unset' -f command 2>/dev/null
'command' -v javac
)" || :

if [ ! -x "${JAVACMD-}" ] || [ ! -x "${JAVACMD-}" ]; then
echo "The java/javac command does not exist in PATH nor is JAVA_HOME set,
so mvnw cannot run." >&2
return 1
fi
fi
}

# hash string like Java String::hashCode
hash_string() {
str="${1:-}" h=0
while [ -n "$str" ]; do
char="${str%${str#?}}"
h=$((h * 31 + $(LC_CTYPE=C printf %d "'$char') % 4294967296))
str="${str#?}"

```

```

done

printf %x\\n $h
}

verbose() { ;; }

[ "${MVNW_VERBOSE-}" != true ] || verbose() { printf %s\\n "${1-}"; }

die() {
    printf %s\\n "$1" >&2
    exit 1
}

trim() {
    # MWRAPPER-139:
    #   Trims trailing and leading whitespace, carriage returns, tabs, and
    #   linefeeds.
    #   Needed for removing poorly interpreted newline sequences when running
    #   in more
    #   exotic environments such as mingw bash on Windows.
    printf "%s" "${1}" | tr -d '[:space:]'
}

scriptDir="$(dirname "$0")"
scriptName="$(basename "$0")"

# parse distributionUrl and optional distributionSha256Sum, requires
# .mvn/wrapper/maven-wrapper.properties
while IFS="=" read -r key value; do
    case "${key-}" in
        distributionUrl) distributionUrl=$(trim "${value-}") ;;
        distributionSha256Sum) distributionSha256Sum=$(trim "${value-}") ;;
        esac
done <"$scriptDir/.mvn/wrapper/maven-wrapper.properties"

[ -n "${distributionUrl-}" ] || die "cannot read distributionUrl property in
    $scriptDir/.mvn/wrapper/maven-wrapper.properties"

case "${distributionUrl##*/}"
in maven-mvnd-*bin.*)
    MVN_CMD=mvnd.sh _MVNW_REPO_PATTERN=/maven/mvnd/
    case "${PROCESSOR_ARCHITECTURE-}${PROCESSOR_ARCHITEW6432-}:${uname -a}" in

```



```

*AMD64:CYGWIN* | *AMD64:MINGW*) distributionPlatform=windows-amd64 ;;
:Darwin*x86_64) distributionPlatform=darwin-amd64 ;;
:Darwin*arm64) distributionPlatform=darwin-aarch64 ;;
:Linux*x86_64*) distributionPlatform=linux-amd64 ;;
*)
    echo "Cannot detect native platform for mvnd on $(uname)-$(uname -m), use
        pure java version" >&2
    distributionPlatform=linux-amd64
    ;;
esac
distributionUrl="${distributionUrl%-bin.*}-${distributionPlatform}.zip"
;;
maven-mvnd-*) MVN_CMD=mvnd.sh _MVNW_REPO_PATTERN=/maven/mvnd/ ;;
*) MVN_CMD="mvn${scriptName#mvnw}" _MVNW_REPO_PATTERN=/org/apache/maven/ ;;
esac

# apply MVNW_REPOURL and calculate MAVEN_HOME
# maven home pattern:
~/ .m2/wrapper/dists/{apache-maven-<version>,maven-mvnd-<version>-<platform>}/<ha
sh>
[ -z "${MVNW_REPOURL-}" ] ||
    distributionUrl="$MVNW_REPOURL$_MVNW_REPO_PATTERN${distributionUrl#*"$_MVNW_REPO
_PATTERN"}"
distributionUrlName="${distributionUrl##*/}"
distributionUrlNameMain="${distributionUrlName%.*}"
distributionUrlNameMain="${distributionUrlNameMain%-bin}"
" MAVEN_USER_HOME="${MAVEN_USER_HOME:-${HOME}/.m2}"
MAVEN_HOME="${MAVEN_USER_HOME}/wrapper/dists/${distributionUrlNameMain-}/${hash_st
ring "$distributionUrl"}"

exec_maven() {
    unset MVNW_VERBOSE MVNW_USERNAME MVNW_PASSWORD MVNW_REPOURL || :
    exec "$MAVEN_HOME/bin/$MVN_CMD" "$@" || die "cannot exec
        $MAVEN_HOME/bin/$MVN_CMD"
}

if [ -d "$MAVEN_HOME" ]; then
    verbose "found existing MAVEN_HOME at $MAVEN_HOME"
    exec_maven "$@"
fi

```

```

case "${distributionUrl-}" in
    *?-bin.zip | *?maven-mvnd-*?*.zip) ;;
    *) die "distributionUrl is not valid, must match *-bin.zip or maven-mvnd-*.zip,
        but found '${distributionUrl-}'" ;;
esac

# prepare tmp dir
if TMP_DOWNLOAD_DIR="$(mktemp -d)" && [ -d "$TMP_DOWNLOAD_DIR" ]; then
    clean() { rm -rf -- "$TMP_DOWNLOAD_DIR"; }
    trap clean HUP INT TERM EXIT
else
    die "cannot create temp dir"
fi

mkdir -p -- "${MAVEN_HOME%/*}"

# Download and Install Apache Maven
verbose "Couldn't find MAVEN_HOME, downloading and installing it ..."
verbose "Downloading from: $distributionUrl"
verbose "Downloading to: $TMP_DOWNLOAD_DIR/$distributionUrlName"

# select .zip or .tar.gz
if ! command -v unzip >/dev/null; then
    distributionUrl="${distributionUrl%.zip}.tar.gz"
    distributionUrlName="${distributionUrl##*/}"
fi

# verbose opt
_MVNW_QUIET_CURL=--silent      _MVNW_QUIET_UNZIP=-q
MVNW_QUIET_WGET=--quiet
_MVNW_QUIET_TAR=''

[ "${MVNW_VERBOSE-}" != true ] || _MVNW_QUIET_CURL=''
_MVNW_QUIET_WGET=''

_MVNW_QUIET_UNZIP='' _MVNW_QUIET_TAR=v

# normalize http auth
case "${MVNW_PASSWORD:+has-password}" in
    '') MVNW_USERNAME='' MVNW_PASSWORD='' ;;
    has-password) [ -n "${MVNW_USERNAME-}" ] || MVNW_USERNAME='' MVNW_PASSWORD='' ;;
esac

```

```

if [ -z "${MVNW_USERNAME-}" ] && command -v wget >/dev/null; then
    verbose "Found wget ... using wget"
    wget ${_MVNW_QUIET_WGET:+"${_MVNW_QUIET_WGET"}"} "$distributionUrl" -O
        "$TMP_DOWNLOAD_DIR/$distributionUrlName" || die "wget: Failed to fetch
        $distributionUrl"
elif [ -z "${MVNW_USERNAME-}" ] && command -v curl >/dev/null; then
    verbose "Found curl ... using curl"
    curl ${_MVNW_QUIET_CURL:+"${_MVNW_QUIET_CURL"}"} -f -L -o
        "$TMP_DOWNLOAD_DIR/$distributionUrlName" "$distributionUrl" || die "curl:
        Failed to fetch $distributionUrl"
elif set_java_home; then
    verbose "Falling back to use Java to download"
    javaSource="$TMP_DOWNLOAD_DIR/Downloader.java"
    targetZip="$TMP_DOWNLOAD_DIR/$distributionUrlName"
    cat >"$javaSource" <<-END
        public class Downloader extends java.net.Authenticator
        {
            protected java.net.PasswordAuthentication getPasswordAuthentication()
            {
                return new java.net.PasswordAuthentication( System.getenv(
                    "MVNW_USERNAME" ), System.getenv( "MVNW_PASSWORD" ).toCharArray() );
            }
            public static void main( String[] args ) throws Exception
            {
                setDefault( new Downloader() );
                java.nio.file.Files.copy( java.net.URI.create( args[0]
                    ).toURL().openStream(), java.nio.file.Paths.get( args[1]
                    ).toAbsolutePath().normalize() );
            }
        }
    END
    # For Cygwin/MinGW, switch paths to Windows format before running javac and java
    verbose " - Compiling Downloader.java ..."
    "$(native_path "$JAVACMD")" "$(native_path "$javaSource")" || die "Failed to
        compile Downloader.java"
    verbose " - Running Downloader.java ..."
    "$(native_path "$JAVACMD")" -cp "$(native_path "$TMP_DOWNLOAD_DIR")" Downloader
        "$distributionUrl" "$(native_path "$targetZip")"
fi

```

```

# If specified, validate the SHA-256 sum of the Maven distribution zip file
if [ -n "${distributionSha256Sum-}" ]; then
    distributionSha256Result=false
    if [ "$MVN_CMD" = mvnd.sh ]; then
        echo "Checksum validation is not supported for maven-mvnd." >&2
        echo "Please disable validation by removing 'distributionSha256Sum' from your
            maven-wrapper.properties." >&2
        exit 1
    elif command -v sha256sum >/dev/null; then
        if echo "$distributionSha256Sum $TMP_DOWNLOAD_DIR/$distributionUrlName" |
            sha256sum -c - >/dev/null 2>&1; then
            distributionSha256Result=true
        fi
    elif command -v shasum >/dev/null; then
        if echo "$distributionSha256Sum $TMP_DOWNLOAD_DIR/$distributionUrlName" |
            shasum -a 256 -c >/dev/null 2>&1; then
            distributionSha256Result=true
        fi
    else
        echo "Checksum validation was requested but neither 'sha256sum' or 'shasum'
            are available." >&2
        echo "Please install either command, or disable validation by removing
            'distributionSha256Sum' from your maven-wrapper.properties." >&2
        exit 1
    fi
    if [ $distributionSha256Result = false ]; then
        echo "Error: Failed to validate Maven distribution SHA-256, your Maven
            distribution might be compromised." >&2
        echo "If you updated your Maven version, you need to update the specified
            distributionSha256Sum property." >&2
        exit 1
    fi
fi

# unzip and move
if command -v unzip >/dev/null; then
    unzip ${_MVNW_QUIET_UNZIP:+"$_MVNW_QUIET_UNZIP"}
        "$TMP_DOWNLOAD_DIR/$distributionUrlName" -d "$TMP_DOWNLOAD_DIR" || die
        "failed to unzip"
else

```

```

tar xzf${_MVNW_QUIET_TAR:+"${_MVNW_QUIET_TAR"}
    "$TMP_DOWNLOAD_DIR/$distributionUrlName" -C "$TMP_DOWNLOAD_DIR" || die
    "failed to untar"
fi

# Find the actual extracted directory name (handles snapshots where filename !=
    directory name)
actualDistributionDir=""

# First try the expected directory name (for regular distributions)
if [ -d "$TMP_DOWNLOAD_DIR/$distributionUrlNameMain" ]; then
    if [ -f "$TMP_DOWNLOAD_DIR/$distributionUrlNameMain/bin/$MVN_CMD" ]; then
        actualDistributionDir="$distributionUrlNameMain"
    fi
fi

# If not found, search for any directory with the Maven executable (for snapshots)
if [ -z "$actualDistributionDir" ]; then
    # enable globbing to iterate over items
    set +f
    for dir in "$TMP_DOWNLOAD_DIR"/*; do
        if [ -d "$dir" ]; then
            if [ -f "$dir/bin/$MVN_CMD" ]; then
                actualDistributionDir="$(basename "$dir")"
                break
            fi
        fi
    done
    set -f
fi

if [ -z "$actualDistributionDir" ]; then
    verbose "Contents of $TMP_DOWNLOAD_DIR:"
    verbose "$(ls -la "$TMP_DOWNLOAD_DIR")"
    die "Could not find Maven distribution directory in extracted archive"
fi

verbose "Found extracted Maven distribution directory: $actualDistributionDir"
printf %s\\n "$distributionUrl"
>"$TMP_DOWNLOAD_DIR/$actualDistributionDir/mvnw.url"

```

```

mv -- "$TMP_DOWNLOAD_DIR/$actualDistributionDir" "$MAVEN_HOME" || [ -d
    "$MAVEN_HOME" ] || die "fail to move MAVEN_HOME"

clean || : exec_maven

"$@"

```

mvnw.cmd

```

<# : batch portion

@REM -----

@REM Licensed to the Apache Software Foundation (ASF) under one
@REM or more contributor license agreements. See the NOTICE file
@REM distributed with this work for additional information
@REM regarding copyright ownership. The ASF licenses this file
@REM to you under the Apache License, Version 2.0 (the
@REM "License"); you may not use this file except in
@REM compliance with the License. You may obtain a copy of the
@REM License at
@REM http://www.apache.org/licenses/LICENSE-2.0
@REM
@REM Unless required by applicable law or agreed to in writing,
@REM software distributed under the License is distributed on an
@REM "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
@REM KIND, either express or implied. See the License for the
@REM specific language governing permissions and limitations
@REM under the License.
@REM -----

@REM -----

@REM Apache Maven Wrapper startup batch script, version 3.3.4
@REM
@REM Optional ENV vars
@REM MVNW_REPOURL - repo url base for downloading maven distribution
@REM MVNW_USERNAME/MVNW_PASSWORD - user and password for downloading maven
@REM MVNW_VERBOSE - true: enable verbose log; others: silence the output
@REM -----

@if "%_MVNW_ARG0_NAME%"==" " (set _MVNW_ARG0_NAME=%~nx0) @set
MVNW_CMD_=
@set _MVNW_ERROR_=
@set _MVNW_PSMODULEP_SAVE=%PSModulePath%

```

```

@SET PSModulePath=

@FOR /F "usebackq tokens=1* delims==" %%A IN (`powershell -noprofile "&
    {$scriptDir='%~dp0'; $script='%_MVNW_ARG0_NAME_'; icm -ScriptBlock
    ([Scriptblock]::Create((Get-Content -Raw '%~f0')) -NoNewScope}"`) DO @(
    IF "%%A"=="MVN_CMD"
        (set _MVNW_CMD_=%%B) ELSE IF "%%B"==" " (echo
        (echo %%A=%%B)
    )
)

@SET PSModulePath=%_MVNW_PSMODULEP_SAVE%
@SET _MVNW_PSMODULEP_SAVE=
@SET _MVNW_ARG0_NAME_=
@SET MVNW_USERNAME=
@SET MVNW_PASSWORD=
@IF NOT "%_MVNW_CMD_"==" " ("%_MVNW_CMD_" %*)
@echo Cannot start maven from wrapper >&2 && exit /b 1
@GOTO :EOF
: end batch / begin powershell #>

$ErrorActionPreference = "Stop"
if ($env:MVNW_VERBOSE -eq "true") {
    $VerbosePreference = "Continue"
}

# calculate distributionUrl, requires .mvn/wrapper/maven-wrapper.properties
$distributionUrl = (Get-Content -Raw
    "$scriptDir/.mvn/wrapper/maven-wrapper.properties"
    | ConvertFrom-StringData).distributionUrl
if (!$distributionUrl) {
    Write-Error "cannot read distributionUrl property in
        $scriptDir/.mvn/wrapper/maven-wrapper.properties"
}

switch -wildcard -casesensitive ( $($distributionUrl -replace '^.*/','') ) {
    "maven-mvnd-*" {
        $USE_MVND = $true
        $distributionUrl = $distributionUrl -replace
            '-bin\[^\.]*$', "-windows-amd64.zip"
        $MVN_CMD = "mvnd.cmd"
        break
    }
    default {

```

```

    $USE_MVND = $false
    $MVN_CMD = $script -replace '^mvnw','mvn'
    break
}
}

# apply MVNW_REPOURL and calculate MAVEN_HOME
# maven home pattern:
~/ .m2/wrapper/dists/{apache-maven-<version>,maven-mvnd-<version>-<platform>}/<ha
sh>

if ($env:MVNW_REPOURL) {
    $MVNW_REPO_PATTERN = if ($USE_MVND -eq $False) { "/org/apache/maven/" } else {
        "/maven/mvnd/" }
    $distributionUrl = "$env:MVNW_REPOURL$MVNW_REPO_PATTERN$($distributionUrl
        -replace "^.*$MVNW_REPO_PATTERN",'') "
    }
    $distributionUrlName = $distributionUrl -replace '^.*/',''
    $distributionUrlNameMain = $distributionUrlName -replace '\.[^.]*$','' -replace
        '-bin$',''

    $MAVEN_M2_PATH = "$HOME/.m2"
    if ($env:MAVEN_USER_HOME) {
        $MAVEN_M2_PATH = "$env:MAVEN_USER_HOME"
    }

    if (-not (Test-Path -Path $MAVEN_M2_PATH)) {
        New-Item -Path $MAVEN_M2_PATH -ItemType Directory | Out-Null
    }

    $MAVEN_WRAPPER_DISTS = $null
    if ((Get-Item $MAVEN_M2_PATH).Target[0] -eq $null) {
        $MAVEN_WRAPPER_DISTS = "$MAVEN_M2_PATH/wrapper/dists"
    } else {
        $MAVEN_WRAPPER_DISTS = (Get-Item $MAVEN_M2_PATH).Target[0] + "/wrapper/dists"
    }

    $MAVEN_HOME_PARENT = "$MAVEN_WRAPPER_DISTS/$distributionUrlNameMain"
    $MAVEN_HOME_NAME =
        ([System.Security.Cryptography.SHA256]::Create().ComputeHash([byte[]][char[]]$di
            stributionUrl) | ForEach-Object {$_.ToString("x2")}) -join ' '

```



```

$MAVEN_HOME = "$MAVEN_HOME_PARENT/$MAVEN_HOME_NAME"

if (Test-Path -Path "$MAVEN_HOME" -PathType Container) {
    Write-Verbose "found existing MAVEN_HOME at $MAVEN_HOME"
    Write-Output "MVN_CMD=$MAVEN_HOME/bin/$MVN_CMD"
    exit $?
}

if (! $distributionUrlNameMain -or ($distributionUrlName -eq
    $distributionUrlNameMain)) {
    Write-Error "distributionUrl is not valid, must end with *-bin.zip, but found
        $distributionUrl"
}

# prepare tmp dir
$TMP_DOWNLOAD_DIR_HOLDER = New-TemporaryFile
$TMP_DOWNLOAD_DIR = New-Item -ItemType Directory -Path
    "$TMP_DOWNLOAD_DIR_HOLDER.dir"
$TMP_DOWNLOAD_DIR_HOLDER.Delete() | Out-Null
trap {
    if ($TMP_DOWNLOAD_DIR.Exists) {
        try { Remove-Item $TMP_DOWNLOAD_DIR -Recurse -Force | Out-Null }
        catch { Write-Warning "Cannot remove $TMP_DOWNLOAD_DIR" }
    }
}

New-Item -ItemType Directory -Path "$MAVEN_HOME_PARENT" -Force | Out-Null

# Download and Install Apache Maven
Write-Verbose "Couldn't find MAVEN_HOME, downloading and installing it ..."
Write-Verbose "Downloading from: $distributionUrl"
Write-Verbose "Downloading to: $TMP_DOWNLOAD_DIR/$distributionUrlName"

$webclient = New-Object System.Net.WebClient
if ($env:MVNW_USERNAME -and $env:MVNW_PASSWORD) {
    $webclient.Credentials = New-Object
        System.Net.NetworkCredential($env:MVNW_USERNAME, $env:MVNW_PASSWORD)
}
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
$webclient.DownloadFile($distributionUrl,

```

```

"$TMP_DOWNLOAD_DIR/$distributionUrlName") | Out-Null

# If specified, validate the SHA-256 sum of the Maven distribution zip file
$distributionSha256Sum = (Get-Content -Raw
    "$scriptDir/.mvn/wrapper/maven-wrapper.properties"
    | ConvertFrom-StringData).distributionSha256Sum
if ($distributionSha256Sum) {
    if ($USE_MVND) {
        Write-Error "Checksum validation is not supported for maven-mvnd. `nPlease
            disable validation by removing 'distributionSha256Sum' from your
            maven-wrapper.properties."
    }
    Import-Module $PSHOME\Modules\Microsoft.PowerShell.Utility -Function
        Get-FileHash
    if ((Get-FileHash "$TMP_DOWNLOAD_DIR/$distributionUrlName" -Algorithm
        SHA256).Hash.ToLower() -ne $distributionSha256Sum) {
        Write-Error "Error: Failed to validate Maven distribution SHA-256, your Maven
            distribution might be compromised. If you updated your Maven version, you
            need to update the specified distributionSha256Sum property."
    }
}

# unzip and move
Expand-Archive "$TMP_DOWNLOAD_DIR/$distributionUrlName" -DestinationPath
    "$TMP_DOWNLOAD_DIR" | Out-Null

# Find the actual extracted directory name (handles snapshots where filename !=
    directory name)
$actualDistributionDir = ""

# First try the expected directory name (for regular distributions)
$expectedPath = Join-Path "$TMP_DOWNLOAD_DIR" "$distributionUrlNameMain"
$expectedMvnPath = Join-Path "$expectedPath" "bin/$MVN_CMD"
if ((Test-Path -Path $expectedPath -PathType Container) -and (Test-Path -Path
    $expectedMvnPath -PathType Leaf)) {
    $actualDistributionDir = $distributionUrlNameMain
}

# If not found, search for any directory with the Maven executable (for snapshots)
if (!$actualDistributionDir) {

```

```

Get-ChildItem -Path "$TMP_DOWNLOAD_DIR" -Directory | ForEach-Object {
    $testPath = Join-Path $_.FullName "bin/$MVN_CMD" if
        (Test-Path -Path $testPath -PathType Leaf) {
    $actualDistributionDir = $_.Name
        }
    }
}

if (!$actualDistributionDir) {
Write-Error "Could not find Maven distribution directory in extracted archive"
}

Write-Verbose "Found extracted Maven distribution directory:
$actualDistributionDir"
Rename-Item -Path "$TMP_DOWNLOAD_DIR/$actualDistributionDir" -NewName
    $MAVEN_HOME_NAME | Out-Null
    try {
Move-Item -Path "$TMP_DOWNLOAD_DIR/$MAVEN_HOME_NAME" -Destination
$MAVEN_HOME_PARENT | Out-Null
    } catch {
        if (!(Test-Path -Path "$MAVEN_HOME" -PathType Container)) {
            Write-Error "fail to move MAVEN_HOME"
        }
    } finally {
try { Remove-Item $TMP_DOWNLOAD_DIR -Recurse -Force | Out-Null } catch
    { Write-Warning "Cannot remove $TMP_DOWNLOAD_DIR" }
}

Write-Output "MVN_CMD=$MAVEN_HOME/bin/$MVN_CMD"

```

pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>

```

```

    <version>3.5.16</version>
    <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.cognizant</groupId>
<artifactId>spring-rest-handson</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name/>
<description/>
<url/>
<licenses>
    <license/>
</licenses>
<developers>
    <developer/>
</developers>
<scm>
    <connection/>
    <developerConnection/>
    <tag/>
    <url/>
</scm>
<properties>
    <java.version>17</java.version>
</properties>
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-devtools</artifactId>
        <scope>runtime</scope>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>

```

```

</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <executions>
                <execution>
                    <id>default-compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <annotationProcessorPaths>
                            <path>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </path>
                        </annotationProcessorPaths>
                    </configuration>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>

```

```

</execution>
<execution>
<id>default-testCompile</id>
<phase>test-compile</phase>
<goals>
<goal>testCompile</goal>
</goals>
<configuration>
<annotationProcessorPaths>
<path>
<groupId>org.projectlombok</groupId>
<artifactId>lombok</artifactId>
</path>
</annotationProcessorPaths>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>

</project>

```

SpringRestHandsonApplication.java

```

package com.cognizant;

import com.cognizant.bean.Country;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication; import
    org.springframework.context.support.ClassPathXmlApplicationContext;

@SpringBootApplication
public class SpringRestHandsonApplication {

    public static void main(String[] args) {

        SpringApplication.run(SpringRestHandsonApplication.class, args);

        ClassPathXmlApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
    }
}

```

```
Country country = context.getBean("country", Country.class);

System.out.println(country);

context.close();
    }
}
```

Country.java

```
package com.cognizant.bean;

public class Country {

    private String code;
        private String name;

    public Country() {
        }

        public Country(String code, String name) {
            this.code = code;
this.name = name;
        }

        public String getCode() {
            return code;
        }

        public void setCode(String code) {
            this.code = code;
        }

        public String getName() {
            return name;
        }

        public void setName(String name) {
            this.name = name;
        }
}
```

```

@Override
public String toString() {
    return "Country [code=" + code + ", name=" + name + "]";
}
}

```

CountryController.java

```

package com.cognizant.controller;

import com.cognizant.bean.Country;
import com.cognizant.service.CountryService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {

    @Autowired
    private CountryService countryService;

    @GetMapping("/country") public
        Country getCountry() {
    return countryService.getCountry();
        }

    @GetMapping("/country/{code}")
        public Country getCountryByCode(@PathVariable String code) {
            return countryService.getCountry(code);
        }
}

```

HelloController.java

```

package com.cognizant.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

```



```

@RestController
public class HelloController {

    @GetMapping("/hello") public
        String sayHello() {
return "Hello World!!";
        }
    }
}

```

CountryService.java

```

package com.cognizant.service;

import com.cognizant.bean.Country;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;

@Service
public class CountryService {

    private List<Country> countries = new ArrayList<>();

    public CountryService() {

        countries.add(new Country("IN", "India")); countries.add(new
            Country("US", "United States")); countries.add(new
            Country("UK", "United Kingdom"));
    }

    public Country getCountry() {
        return countries.get(0);
    }

    public Country getCountry(String code) {

        for (Country country : countries) {
            if (country.getCode().equalsIgnoreCase(code)) {
                return country;
            }
        }
    }
}

```

```

        }

return null;

    }

}

```

application.properties

```
spring.application.name=spring-rest-handson
```

country.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="country" class="com.cognizant.bean.Country">

<property name="code" value="IN"/>

<property name="name" value="India"/>

</bean>

</beans>

```

SpringRestHandsonApplicationTests.java

```

package com.cognizant;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SpringRestHandsonApplication {

    public static void main(String[] args) {
        SpringApplication.run(SpringRestHandsonApplication.class, args);
    }

}

```

```
}
```

application.properties

```
spring.application.name=spring-rest-handson
```

SpringRestHandsonApplication.class

```
[Binary file - content not shown]
```

Country.class

```
[Binary file - content not shown]
```

CountryController.class

```
[Binary file - content not shown]
```

HelloController.class

```
[Binary file - content not shown]
```

CountryService.class

```
[Binary file - content not shown]
```

Country.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">

<bean id="country" class="com.cognizant.bean.Country">

<property name="code" value="IN"/>

<property name="name" value="India"/>

</bean>

</beans>
```

Outputs :

1.

Tomcat started on port 8083 (http) with context path '/'

Started JwtHandsonApplication in ... seconds

GET <http://localhost:8083/authenticate>

Username : user

Password : pwd

{

 "token":

"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJlc2VyIiwiaWF0IjoxNzE5NTQ2MzAwLCJleHAiOjE3MTk1NDc1MDB9.XXXXXXXXXXXXXX"

}

2.Tomcat started on port 8080 (http) with context path '/'

Started SpringRestHandsonApplication in ... seconds

GET <http://localhost:8080/countries>

[

{

 "code": "IN",

 "name": "India"

},

{

 "code": "US",

 "name": "United States"

}

]

{

 "code": "IN",

 "name": "India"

}